

Software Architecture and Design Specification

Project: Web Based ML Model Evaluator

Version: 1.0

Authors: Team 16 – Parigna P V, Reia S, Divakar K S, Nikhil K R

Date: 04-10-2026

Status: Draft

Revision History

Version | Date | Author | Change Summary

Approvals

Role | Name | Signature/Date

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the ATM System.

1.2 Scope

Covers ATM user services: cash withdrawal, balance inquiry, and fund transfer.

1.3 Audience

Developers, QA engineers, security auditors, instructors, and maintenance teams.

1.4 Definitions

ATM, PIN, ADR, PCI-DSS, TLS.

2. Document Overview

2.1 How to use this document

Provides architectural deliverables including UML diagrams, ADRs, threat models, and API design.

2.2 Related Documents

SRS, STP, RTM.

3. Architecture

3.1 Goals & Constraints

Goals: secure, reliable, 99.9% availability. Constraints: PCI-DSS compliance, ATM hardware limits.

3.2 Stakeholders & Concerns

Customers: security, availability

Bank Ops: maintainability

Regulators: compliance

Developers: modularity.

3.3 Component (UML) Diagram

UML Component Diagram: User → Web Frontend → Backend API → Authentication, Dataset, Model, Evaluation and Experiment Services → ML Engine / Database.

3.4 Component Descriptions

- Web Frontend: user screens, dataset selection and result display.
- Authentication Service: login and access control.
- Dataset Service: upload, validation, profiling and preprocessing.
- Model Service: supported models and parameters.
- ML Engine: training, prediction and evaluation.
- Experiment Service: history, comparison and reports.
- Database: users, experiments, results and logs.

3.5 Chosen Architecture Pattern and Rationale

Layered architecture is chosen because the project is small enough for a single application while still having clear responsibilities. Microservices were not selected because they would add unnecessary deployment and communication complexity.

3.6 Technology Stack & Data Stores

Frontend: JavaScript/React or equivalent. Backend: Python/Django or JavaScript/MERN as selected by Team 16. ML: Python with scikit-learn or equivalent. Data processing: pandas/NumPy or equivalent. Database: relational or document database selected by the team. Communication: REST/JSON and HTTPS.

3.7 Risks & Mitigations

Risk: large datasets or long training → Mitigation: controlled sample datasets for normal tests and status/background handling for longer jobs.

Risk: library/version mismatch → Mitigation: document and pin dependency versions.

Risk: invalid uploads → Mitigation: validate file type, size and contents before processing.

3.8 Traceability to Requirements

ML-F-001 (Login) → Authentication Service

ML-F-002/F-003 (Upload/validation) → Dataset Service

ML-F-010 (Training) → Model Service / ML Engine

ML-F-012/F-013 (Evaluation) → Evaluation Service

ML-F-015/F-016/F-017/F-018 → Experiment Service.

3.9 Security Architecture

STRIDE: Spoofing → authentication; Tampering → server-side validation; Information Disclosure → HTTPS and access control; Denial of Service → input validation/rate limiting; Elevation of Privilege → role-based authorization.

4. Design

4.1 Design Overview

The design separates the presentation, API, service, ML processing and data layers. This makes the workflow easier to test and allows new models to be added without changing the whole application.

4.2 UML Sequence Diagrams

At least 2 sequence diagrams covering 2 flows specific to your project

Sequence Diagram 1 – Dataset Upload: User → Frontend → Dataset API → Dataset Service → Validation/Preprocessing → Database → Frontend.

Sequence Diagram 2 – Training/Evaluation: User → Frontend → Training API → Model Service → ML Engine → Evaluation → Experiment Service → Database → Frontend.

4.3 API Design

Interface definition for at least 2 components.

Endpoint: /withdraw

Method: POST

Request: {cardId, pin, amount}

Response: {status, balance}

Errors: 401 Invalid PIN, 402 Insufficient funds.

4.4 Error Handling, Logging & Monitoring

Use clear error messages and suitable HTTP status codes. Do not write passwords or secrets to logs. Logs should include timestamps, actions and errors. Monitor API failures, training failures, response time and application availability.

4.5 UX Design

The UI should have clear navigation and readable text. Main flow: Dataset → Preprocessing → Model Selection → Training → Evaluation → Comparison → Report. Progress and validation messages should be shown.

4.6 Open Issues & Next Steps

Possible future enhancements include more ML algorithms, hyperparameter tuning, cross-validation, automated model recommendation, explainable AI features and cloud deployment.

5. Appendices

5.1 Glossary: ATM, PIN, PCI-DSS, TLS, ADR.

5.2 References: IEEE 42010, OWASP, NIST SP 800-160.

5.3 Tools: PlantUML, draw.io, Swagger.

SRS Requirement	Architecture / Design Component	STP Test
ML-F-001 Login	Authentication Service	TC-AUTH-01
ML-F-002/F-003 Upload & validation	Dataset Service	TC-DATA-01 / TC-DATA-02
ML-F-006/F-007 Preprocessing & split	Preprocessing Service	TC-PREP-02 / TC-PREP-03
ML-F-008/F-009/F-010 Model & training	Model Service + ML Engine	TC-MODEL-01 / TC-TRAIN-01
ML-F-012/F-013 Evaluation	Evaluation Service	TC-EVAL-01 / TC-EVAL-02
ML-F-015/F-016/F-017 Comparison/history	Experiment Service + Database	TC-COMP-01 / TC-HIST-01
ML-F-018 Report	Report Service	TC-REP-01
ML-SR-002/SR-003 Security	Authentication + Security Layer	TC-SEC-02 / TC-SEC-03

UML Diagrams

UML Component Diagram — Web Based ML Model Evaluator

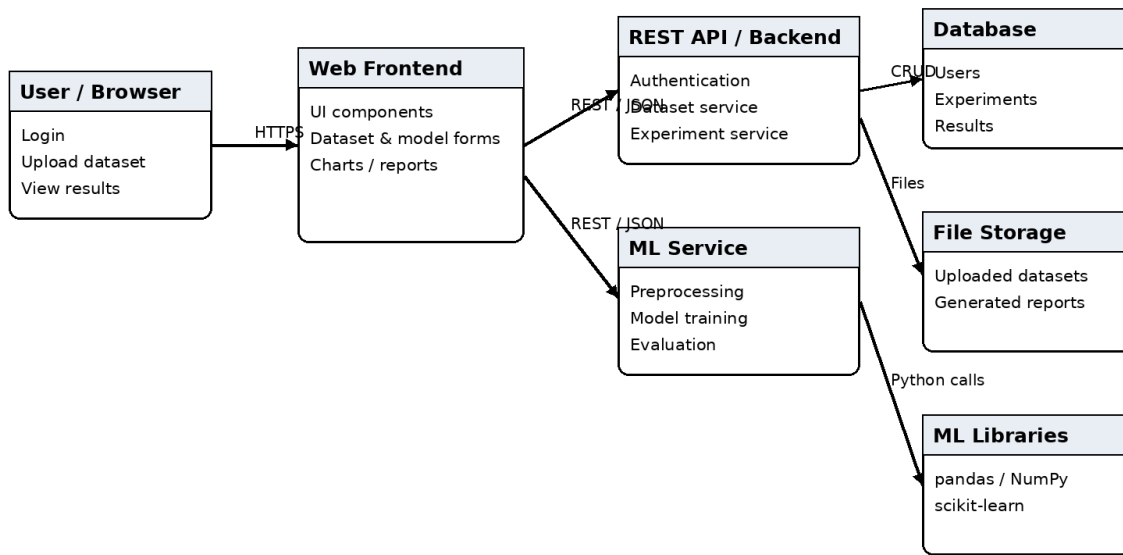


Figure 1. UML Component Diagram

The component diagram shows the major system components and their interfaces.

Sequence Diagram 1: Dataset Upload and Preprocessing

UML Sequence Diagram 1 — Dataset Upload and Preprocessing

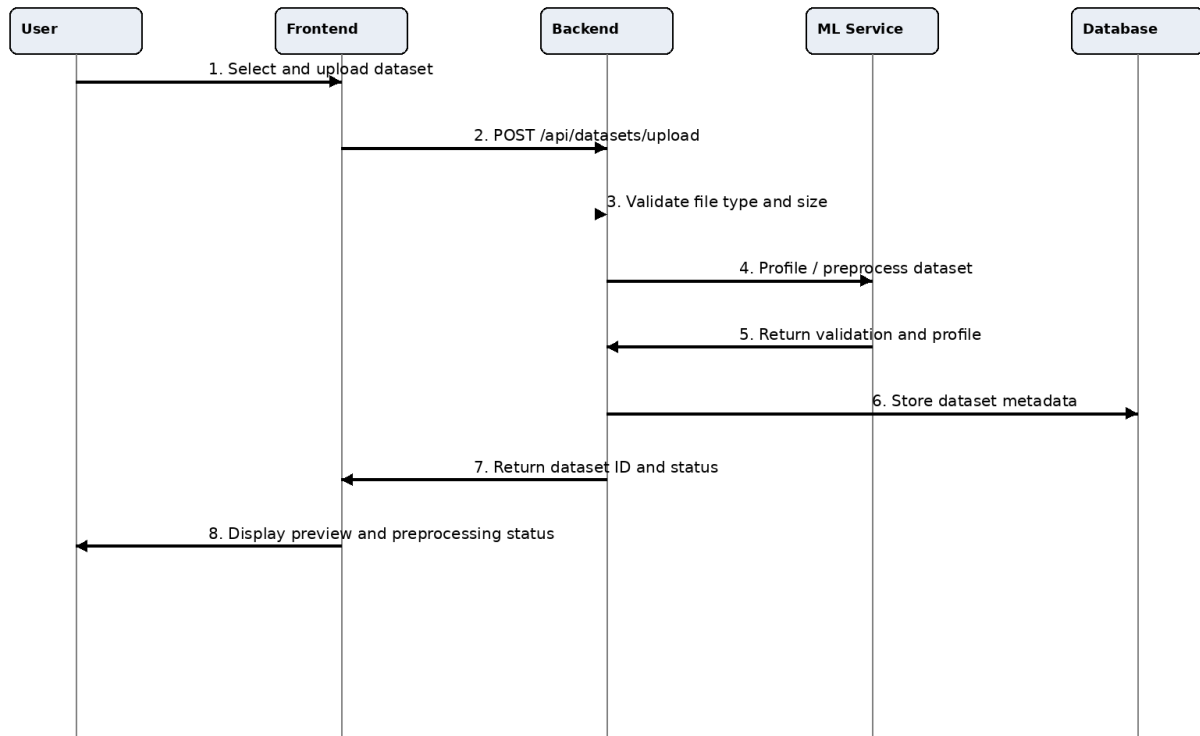


Figure 2. UML Sequence Diagram — Dataset Upload and Preprocessing

Sequence Diagram 2: Model Training and Evaluation

UML Sequence Diagram 2 — Model Training and Evaluation

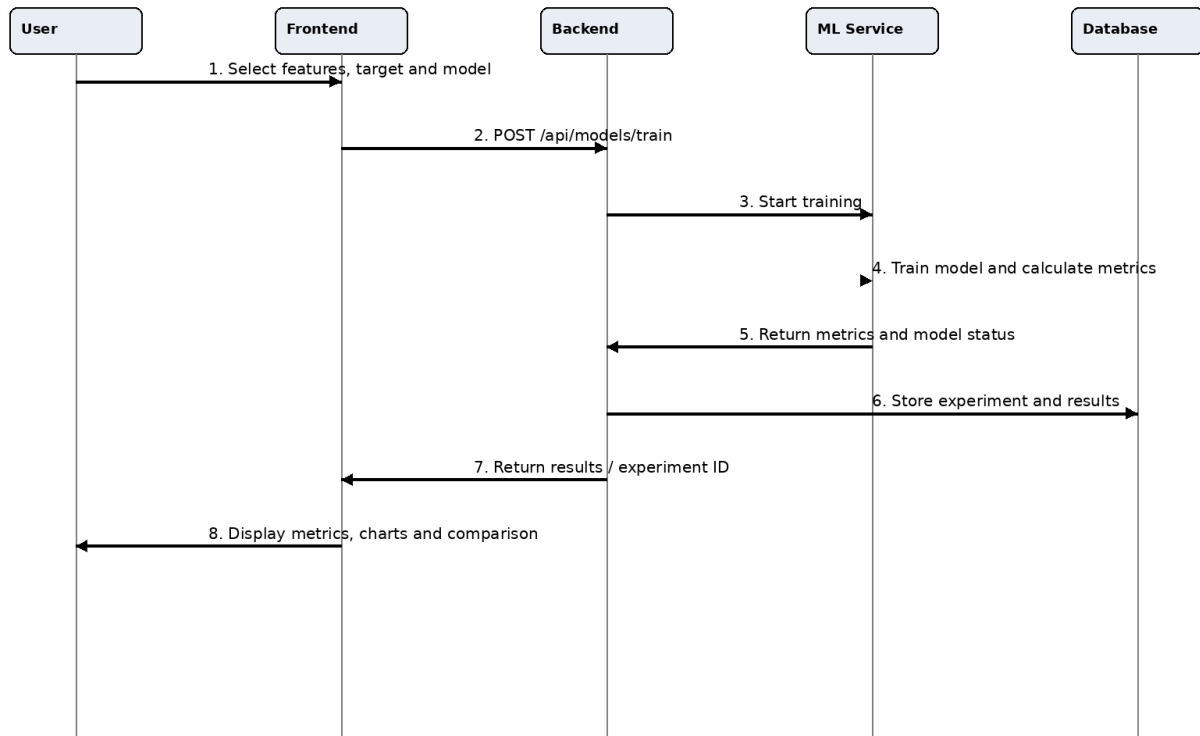


Figure 3. UML Sequence Diagram — Model Training and Evaluation